

```

import streamlit as st
from PyPDF2 import PdfReader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_community.vectorstores import FAISS
from langchain_community.embeddings import OpenAIEmbeddings
from langchain.chains.question_answering import load_qa_chain
from langchain_community.llms import OpenAI
import os

# — Page Config —
st.set_page_config(
    page_title="PDF ChatBot 🤖",
    page_icon="🤖",
    layout="centered"
)

st.title("📄 PDF ChatBot – RAG Project")
st.markdown("Upload any PDF and ask questions from it!")

# — API Key —
api_key = st.text_input("🔑 Enter your OpenAI API Key", type="password")

if api_key:
    os.environ["OPENAI_API_KEY"] = api_key

# — PDF Upload —
pdf = st.file_uploader("📁 Upload your PDF", type="pdf")

if pdf is not None:
    # Step 1: Read PDF
    pdf_reader = PdfReader(pdf)
    text = ""
    for page in pdf_reader.pages:
        text += page.extract_text()

    st.success(f"✅ PDF loaded! {len(pdf_reader.pages)} pages found.")

    # Step 2: Split into chunks
    text_splitter = RecursiveCharacterTextSplitter(
        chunk_size=1000,
        chunk_overlap=200
    )
    chunks = text_splitter.split_text(text)
    st.info(f"📦 Split into {len(chunks)} chunks.")

    # Step 3: Create Vector Store (FAISS)
    with st.spinner("🔄 Building knowledge base..."):
        embeddings = OpenAIEmbeddings()

```

```
vector_store = FAISS.from_texts(chunks, embeddings)
st.success("✅ Knowledge base ready!")
```

```
# Step 4: Ask Questions
```

```
st.markdown("---")
```

```
st.subheader("🗨️ Ask anything from your PDF")
```

```
question = st.text_input("Your Question:")
```

```
if question:
```

```
    with st.spinner("🤖 Thinking..."):
```

```
        # Find relevant chunks
```

```
        docs = vector_store.similarity_search(question, k=3)
```

```
        # Generate answer
```

```
        llm = OpenAI(temperature=0)
```

```
        chain = load_qa_chain(llm, chain_type="stuff")
```

```
        answer = chain.run(input_documents=docs, question=question)
```

```
st.markdown("### 🗨️ Answer:")
```

```
st.write(answer)
```

```
# Show source chunks
```

```
with st.expander("📖 Source chunks used:"):
```

```
    for i, doc in enumerate(docs):
```

```
        st.markdown(f"**Chunk {i+1}:**")
```

```
        st.write(doc.page_content)
```

```
        st.markdown("---")
```

```
else:
```

```
    st.warning("⚠️ Please enter your OpenAI API Key to continue.")
```

```
# — Footer —
```

```
st.markdown("---")
```

```
st.markdown("Built with ❤️ using LangChain + FAISS + Streamlit")
```